Simulador didático de sistemas operacionais

Relatório técnico da implementação em Python orientado a objetos.
Disciplina: Sistemas Operacionais. Professor: Mauricio Acconcia Dias.
Data da verificação: 31/10/2026. Ambiente: Windows, Python 3.14.
Integrantes: preencher pelo grupo antes da entrega acadêmica.

1. Objetivo e escopo

A aplicação reproduz mecanismos de um sistema operacional em um ambiente controlado: processos, threads, escalonamento da CPU, paginação, entrada e saída e gerenciamento de arquivos. A execução é por terminal. Processos e threads são objetos do modelo, sem criação de processos ou threads reais para executar a carga.

A implementação segue o documento especificacoes.pdf até o gerenciamento de arquivos, incluindo os registros, métricas, testes e experimentos necessários para observar os mecanismos. Os dados dos arquivos virtuais, as tabelas e os registros de execução ficam exclusivamente em memória. Não são criados arquivos de dados no computador hospedeiro.

As cargas JSON são entradas estáticas externas ao código, apenas lidas, como exige o enunciado. Também existe entrada por stdin para fornecer uma carga sem criar arquivo adicional. A aplicação não persiste resultados; imprime o log e as métricas no terminal. Documentação e este relatório são artefatos da entrega, separados dos dados simulados.

2. Arquitetura orientada a objetos

Simulador coordena os mecanismos por composição. AgendaEventos mantém um heap de Evento, ordenado pelo instante e pela sequência de inserção. RegistroEventos armazena o log em RAM. Configuracao é um objeto imutável com validação de parâmetros. A interface e a leitura de cenários ficam em módulos separados do núcleo.

Processo reúne um PCB e suas Threads. O PCB guarda identificador, prioridade, chegada, estado agregado, recursos compartilhados, espaço lógico, IDs das threads, descritores e o último contexto observado ao avançar uma instrução. Cada Thread possui um TCB com estado, PC, registradores, pilha lógica, CPU restante e contadores necessários às métricas. A pilha lógica registra o quadro de execução e as chamadas ao sistema de arquivos; não pretende representar a pilha binária de uma arquitetura real.

Escalonador mantém a fila de prontos. PoliticaEscalonamento define uma interface abstrata de seleção. FCFS seleciona a cabeça, RoundRobin reutiliza essa ordem e recebe limitação de quantum no núcleo, e Prioridades seleciona o menor valor numérico. Essa separação permite modificar a ordem sem alterar dispositivos ou arquivos. A herança fica restrita à relação entre políticas; os recursos do SO usam composição.

GerenciadorMemoria administra tabelas e molduras. Dispositivo possui fila e conclusão por evento. SistemaArquivos administra Diretorio, Arquivo, FCB, Bloco e ArquivoAberto. Não há uma única classe que concentre toda a implementação. O diagrama de classes atualizado encontra-se em docs/ARQUITETURA.md.

3. Clock, estados e decisões de execução

O clock é lógico e começa em zero. O programa avança diretamente para o próximo evento, sem aguardar tempo real. Eventos incluem chegada, fim de contexto, fim de intervalo CPU, interrupção de E/S e carregamento de página. Todos os eventos de um instante são processados antes de um novo despacho. Empates seguem a inserção na agenda; admissões seguem chegada e ID do processo, com threads ordenadas por ID.

A thread percorre novo, pronto, executando, bloqueado e finalizado. As transições são validadas: admissão coloca em pronto; despacho coloca em execução; preempção devolve à fila; falta ou E/S bloqueante bloqueia; conclusão do serviço torna pronta; fim das instruções finaliza. Cada mudança registra causa e clock. O estado do processo é derivado das suas threads, priorizando executando, pronto e bloqueado. Todas finalizadas encerram o processo, liberam molduras e fecham descritores.

Somente instruções cpu consomem CPU útil. Operações de controle, tradução, submissão de E/S e chamadas de arquivo são instantâneas, enquanto suas latências configuráveis são tratadas por eventos. Essa escolha deixa o cálculo verificável a mão e explicita quais tempos fazem parte do modelo. A camada de arquivos não aciona automaticamente o dispositivo disco; a carga deve informar es quando quiser representar essa latência.

Prioridades é não preemptiva. FCFS também conserva a CPU até bloqueio ou fim. RR limita CPU útil por quantum, acumulando entre surtos adjacentes. Após o quantum, instruções instantâneas podem terminar ou bloquear o programa; se houver mais CPU, a thread vai ao fim da fila. Uma chegada de maior prioridade não interrompe a thread em execução.

Cada despacho conta como troca de contexto, inclusive o primeiro e a reeleição da mesma thread. O custo é configurável. Nesse intervalo, não há execução útil. A espera de fila termina no despacho; por isso o custo afeta resposta e retorno, mas não é incorporado artificialmente à espera. CPU restante, PC e registradores permanecem no TCB durante preempções e bloqueios.

4. Paginação e entrada e saída

Cada processo tem espaço de endereçamento independente, com tabela de páginas indicando presença e moldura. A tradução usa pagina = endereco // tamanho_pagina, deslocamento = endereco % tamanho_pagina e fisico = moldura * tamanho_pagina + deslocamento. Endereços fora do espaço lógico são rejeitados antes da execução da carga.

Um acesso residente é acerto. Um acesso não residente conta uma falta e bloqueia a thread. Um paginador serial atende em FCFS com tempo configurável, independente dos dispositivos. Na conclusão, usa moldura livre ou remove a página de carregamento mais antigo, em FIFO global. Acertos não atualizam a ordem. A tabela da vítima é invalidada e a nova associação fica registrada.

Faltas simultâneas para a mesma página do mesmo processo compartilham um carregamento. Cada referência ausente continua contando uma falta, portanto faltas e carregamentos não são necessariamente iguais. A referência pendente é concluída no carregamento; a thread não precisa repeti-la no despacho seguinte. Assim, uma substituição intermediária não cria tentativas artificiais nem impede o progresso com poucas molduras.

Há dispositivo de bloco, disco, e de caractere, terminal. Cada um tem sua fila FCFS, estado de atendimento, serviço e métricas. Uma solicitação bloqueante libera a CPU; a conclusão produz interrupção e retorno à fila de prontos. Outra thread do mesmo processo pode executar durante o bloqueio.

E/S não bloqueante devolve o controle imediatamente. Ao terminar, entrega notificação no TCB e registra es_N como concluida. Não há espera ativa. Solicitações pendentes continuam até concluir, mesmo se a thread já finalizou. O horizonte global da execução inclui essa drenagem, distinguindo-se do retorno individual dos processos.

5. Sistema de arquivos em memória

A raiz contém diretórios e arquivos. Cada objeto tem FCB com ID, nome, tipo, tamanho, permissões, criação, modificação e primeiro bloco. Diretórios mantêm entradas por nome; arquivos guardam conteúdo em blocos de tamanho fixo, ligados pelo campo proximo. Diretórios não consomem os blocos reservados ao conteúdo dos arquivos.

O sistema oferece criar diretório e arquivo, abrir, ler, escrever, fechar, remover e listar. A abertura retorna descritor local ao processo e cria entrada na tabela global com referência ao arquivo, modo e cursor. Diferentes processos podem ter descritor zero apontando para aberturas distintas. Cada abertura mantém seu cursor; threads do mesmo processo compartilham a tabela de descritores.

Os modos são r, w, rw e a. w trunca na abertura; rw preserva conteúdo; a escreve ao final. Conteúdo é codificado em UTF-8, e tamanho e cursor são medidos em bytes. Permissões básicas r/w são verificadas na abertura e no uso; diretórios controlam travessia/listagem e alteração de entradas. Não há usuários ou grupos.

São recusados descritores fechados, acesso incompatível, caminhos inexistentes, remoção de arquivo aberto e de diretório não vazio. Falta de espaço é verificada antes da alteração: uma escrita recusada preserva dados e cursor. Erros das chamadas simuladas são registrados e retornam no registrador erro_arquivo; não interrompem o núcleo.

A alocação encadeada permite aproveitar blocos livres não contíguos. Sua limitação é a necessidade de percorrer a cadeia para acessar conteúdo e o custo adicional dos elos. Esta implementação recompõe os dados em RAM ao escrever, favorecendo clareza em vez de eficiência para arquivos extensos. Não há cache, journaling ou persistência física.

6. Métricas e metodologia

Retorno é finalização menos chegada. Espera soma os intervalos na fila de prontos. Resposta vai da chegada à primeira CPU útil; sem CPU, é indefinida e apresentada como n/a. A espera do processo é explicitamente a soma das esperas das threads; sua resposta usa a primeira execução útil entre elas.

Utilização da CPU é CPU útil dividida pelo horizonte global. A sobrecarga é informada separadamente; ociosidade é o restante do horizonte. Utilização de dispositivo considera apenas serviço, sem tempo de fila. Throughput mede processos finalizados por unidade de tempo global. Taxa de faltas divide referências não residentes pelo total de referências. As razões usam zero quando o denominador é zero.

Os experimentos são reproduzidos por python main.py --experimentos. Cada rodada reconstrói todos os objetos e imprime a configuração completa. Não há sorteio nas cargas; a semente 42 é registrada. Comparações internas mantêm as variáveis constantes, mudando somente o fator indicado. As três comparações usam cenários diferentes e seus valores absolutos não devem ser comparados como se fossem uma única carga.

7. Experimento 1: políticas na mesma carga

Carga mista: três processos, quatro threads, 22 unidades de CPU útil, cinco referências de memória, E/S de disco e terminal. Configuração: quantum 2, contexto 0, página 16 bytes, três molduras, falta 3, disco 4, terminal 2; demais capacidades padrão.

Política	Horizonte	CPU útil (%)	Despachos	Espera média/thread	Resposta média/thread
FCFS	26	84,62	11	6,00	6,50
RR q=2	27	81,48	17	8,75	3,75
Prioridades	28	78,57	11	6,00	6,50

Todas as políticas registraram cinco faltas de página. RR antecipou a primeira CPU de threads que aguardariam surtos maiores, reduzindo resposta média de 6,50 para 3,75. Isso não reduziu espera total: a rotação e as interações com bloqueios aumentaram a espera média nesta carga.

Mesmo com contexto gratuito e CPU útil idêntica, os horizontes diferem. A ordem de execução muda o instante das solicitações, o aproveitamento do paralelismo lógico entre CPU/dispositivos e a existência de períodos ociosos. Prioridades não trouxe o menor tempo global. Esses resultados descrevem este cenário, sem estabelecer uma política universalmente melhor.

8. Experimento 2: efeito do quantum

Carga cpu: três processos, surtos de 12, 5 e 8, chegadas em 0, 1 e 2. RR nas duas rodadas, custo por contexto 1; apenas o quantum muda. Não há E/S ou memória; CPU útil total permanece 25.

Quantum	Horizonte	Despachos	Sobrecarga	CPU útil (%)	Resposta média/thread
1	50	25	25	50,00	2,00
4	32	7	7	78,13	5,00

Quantum pequeno permite resposta inicial mais rápida, mas realiza 25 despachos e gasta metade do horizonte em contexto. Quantum 4 reduz 18 despachos, levando a sobrecarga de 25 para 7 e o horizonte de 50 para 32. Em troca, a resposta média cresce de 2 para 5. A comparação mostra o compromisso entre interatividade e aproveitamento da CPU quando a troca tem custo.

9. Experimento 3: capacidade da memória

Carga memoria: um processo, uma thread, sequência de páginas 0, 1, 2, 0, 1, 2, 3, 0, 1, 2, 3. Após cada referência há uma unidade de CPU, totalizando 11. FIFO, página 16, serviço 3, RR q=2 e contexto 0. Apenas o número de molduras muda.

Molduras	Referências	Faltas	Acertos	Horizonte	CPU útil (%)
2	11	11	0	44	25,00
4	11	4	7	23	47,83

Com duas molduras, a alternância remove páginas antes de sua reutilização e toda referência falta. O tempo é 11 + 11 x 3 = 44. Quatro molduras acomodam as quatro páginas: apenas os primeiros acessos faltam, dando 11 + 4 x 3 = 23. A redução de sete faltas elimina 21 unidades de bloqueio. CPU útil não mudou; sua proporção cresce porque o horizonte diminui. Isso não prova monotonicidade geral de FIFO, que pode apresentar anomalias em outras sequências.

10. Verificação e cargas de demonstração

Foram executados 32 testes automatizados com unittest, todos aprovados. O comando é python -B -m unittest discover -s testes -v. A integração executa seis cargas nas três políticas, verificando finalização, limpeza das aberturas, monotonicidade do log e conservação entre CPU útil, contexto e ociosidade.

Os testes incluem ordenação estável de eventos, prioridades não preemptivas, quantum acumulado, métricas conferíveis manualmente, preservação de contexto, tradução, FIFO distinto de LRU, falta compartilhada, moldura única entre processos, E/S independente por dispositivo, notificação assíncrona, descritores locais, permissões, remoção, truncamento, UTF-8 e disco cheio sem corrupção. Há teste de repetição integral de log e métricas.

Um teste bloqueia builtins.open e io.open durante a simulação de arquivos, comprovando que as operações virtuais não dependem do sistema de arquivos hospedeiro. A leitura inicial da carga ocorre antes desse bloqueio, por ser entrada estática autorizada.

A carga verificacao permite conferência manual: P1 CPU 3 e P2 CPU 2, chegadas zero. FCFS termina P1 em 3 e P2 em 5. RR q=2 termina P2 em 4 e P1 em 5. Prioridades executa P2 primeiro, terminando em 2 e P1 em 5. O horizonte é 5 nas três políticas sem custo de contexto.

A carga arquivos demonstra todas as operações e três falhas intencionais: leitura após fechamento, abertura para escrita sem permissão e remoção de caminho inexistente. A contagem esperada é três erros simulados, sem falha do programa. As cargas cpu, es e mista atendem aos perfis mínimos solicitados.

11. Limitações e materiais da entrega

O modelo tem uma CPU lógica, prioridades sem envelhecimento, paginação FIFO global e paginador próprio. Instruções de controle e chamadas de arquivo não gastam CPU útil. Não modela proteção por usuário, memória contígua, multiprocessamento, TLB, LRU, escalonamento de disco avançado, cache ou journaling. Esses limites estão documentados e os recursos opcionais não são declarados como implementados.

Todos os eventos ficam em RAM; cenários extensos podem consumir memória considerável. A tabela de blocos é pequena e usa varredura por clareza didática. Leituras parciais podem dividir caracteres UTF-8 na exibição; os bytes do arquivo permanecem corretos. Métricas de E/S usam o horizonte que inclui solicitações assíncronas após o fim dos processos.

A entrega contém código-fonte, seis cenários JSON, testes, README, diagrama, guia de apresentação e relatório em Markdown/PDF. A publicação no GitHub, identificação dos integrantes e formação do histórico real de participação devem ser feitas pelo grupo; não foram criados repositório remoto nem commits artificiais.

12. Fontes e ferramentas

Fonte dos requisitos: especificacoes.pdf, Especificação do trabalho prático: Simulador didático de sistemas operacionais, Prof. Mauricio Acconcia Dias, fornecido junto ao projeto.

Bibliotecas utilizadas: biblioteca padrão Python, principalmente dataclasses, enum, abc, heapq, collections, itertools, json, argparse, pathlib e unittest. Não foram utilizadas bibliotecas que implementam escalonamento, paginação ou sistema de arquivos do simulador.

O desenvolvimento foi assistido por OpenAI Codex. A leitura do PDF usou reconhecimento de texto do Windows como ferramenta de apoio; o programa entregue não depende dele. O grupo deve identificar essa assistência conforme as regras da disciplina e compreender as decisões, os testes e as limitações antes de apresentar a solução.